COMPUTER SCIENCE + COMPUTER ENGINEERING

A guided undergraduate-to-graduate course for games

09

Gate H: Hardware/software co-design

Workload, architecture, verification and design review

PACKAGE 09 / VOLUME 08 / SEPTEMBER 28, 2026
Modules 36–40 · Gate H · Expanded teaching edition 1.0
Study, implement, test and review. Models are not physical measurements.

Contents

The decision to be defended 3
Entry requirements and acceptable routes 3
Workload dossier 3
Requirement contract 4
Baseline fairness 5
Architecture alternatives 5
Numerical verification 5
Host/device interface and failure plan 6
Evaluation matrix 6
Sensitivity and application benefit 7
Milestones and review artifacts 7
Suggested portfolio layout 7
Rubric and mandatory acceptance conditions 8
Oral review prompts 8
Boundaries of the supplied reference 9

The decision to be defended

Choose one measured game-engine workload and determine whether a specialized architecture improves the complete useful operation under an explicit quality and platform contract. A well-supported recommendation to keep or improve software is a valid outcome. A fast isolated arithmetic model with no workload evidence is not the capstone.

The original Gate H requirement is reproduced below without changing its scope:

After Volume 8. Choose one measured engine workload. Compare a competent software baseline with at least two architecture alternatives, implementing one alternative as RTL, a supported simulation model or a bounded analytical prototype with clear limitations.

Required evidence includes workload distributions, numerical-quality limits, host/device ownership, setup and transfer costs, resource/timing estimates, failure handling and a sensitivity analysis. Evaluate small and large workloads and one adverse case. A negative conclusion is acceptable: proving that a software optimization is preferable is valid co-design work.

The review must identify the maximum application benefit, the dominant bottleneck, which results are modeled and which are measured, and what changes under a tighter memory or latency constraint. Do not compare an optimized accelerator against an intentionally weak software baseline.

Entry requirements and acceptable routes

Complete Gates F and G and Modules 36–39, or document equivalent evidence reviewed against their outcomes. Bring an engine workload, a functioning software baseline and enough instrumentation to identify its useful-work boundary. Gate H is an upper-undergraduate computer-engineering checkpoint, not a production tape-out, degree or license.

Three implementation routes are acceptable: an RTL design with reproducible simulation and appropriate synthesis evidence; a supported architectural/systems simulation with a stated model configuration; or a bounded analytical/behavioral prototype with explicit limits. All routes still require a measured software workload and a defensible baseline. A Python model can implement the specialized behavior without pretending its own interpreter runtime is accelerator runtime.

Physical deployment is optional. A schematic, FPGA board or measured device can deepen evidence when properly reviewed, but neither new equipment nor proprietary platform access is a prerequisite. Do not turn a model's assumed frequency into a physical claim simply because the optional hardware route is not used.

Workload dossier

Identify the engine subsystem, operation and actual reason it might need acceleration. Suitable candidates include a bounded image operation, transform batch, visibility predicate, audio block, decompression substep or controller-processing path. These are examples, not preapproved workload choices. Avoid a problem so broad that no trusted reference can be built or a problem so trivial that setup dominates every plausible use without a meaningful investigation.

Record scene or replay source, engine/build version, input format, batch distribution, bytes moved, arithmetic, dependencies and the frame or interaction contribution. Retain small, typical, large and adverse datasets. Protect private or licensed assets; include an authorized reproducible substitute when necessary and explain differences. Record how many observations support the distribution instead of asserting that one benchmark is representative.

The supplied affine example is synthetic. To use it as a learner project, identify an actual engine operation with that or a justified related contract, collect its workload distribution and quality requirements, and adapt the example. Replaying the bundled six sizes without an engine connection demonstrates the example, not the required workload characterization.

Requirement contract

Assign requirement IDs before collecting comparative results. Every performance requirement needs a metric, unit, boundary, workload, operating condition and acceptance method. Hard conditions include correct outputs under the approved numerical contract, no out-of-range or wrong-context access, no premature buffer reuse and defined bounded failure handling. Performance and resource goals may produce a negative result; hard correctness conditions may not be hidden inside a weighted average.

Requirement family	Required definition	Evidence
Functional	Oracle, inputs, outputs and exceptional cases	Exact or tolerance-aware comparisons
Numerical	Range, precision, rounding, saturation and quality threshold	Boundary and representative error results
Latency	Start/end events, clocks, queueing and tail statistic	Raw traces or conditional model with limits
Throughput	Useful work per elapsed interval, steady/batch distinction	Validated repeated workload
Memory	Payload, workspace, old/new overlap and ownership	Peak budget and failure tests
Failure	Timeout, cancellation, reset, removal and fallback	State-machine traces
Resource	Clock, lanes, memory, logic or physical scope	Assumption, synthesis or measurement tier

Do not borrow threshold values from the teaching example as if they were product requirements. Define them from the game, target platform and acceptable quality. Document what result would falsify the proposed benefit.

Baseline fairness

Use an appropriate algorithm and implementation for the target. Record compiler version/options, language/runtime, layout, vectorization evidence when claimed, batching, allocations and synchronization. Compare equal useful results and quality. Include a credible existing CPU or GPU path where it materially changes the decision. A deliberately slow interpreter, disabled optimization or omitted batching is not a fair comparator for a modeled optimized device.

Keep the correctness oracle separate from the timed baseline. Validate full results on boundary/adversarial cases and representative samples; use checksums only as an additional guard. A no-op or wrong-result variant must be rejected by the harness. Report all timing boundaries so a kernel-only number cannot be mistaken for application latency.

Architecture alternatives

Compare the baseline with at least two architecture alternatives, and implement at least one specialized route. Examples are a single-lane pipeline and a multi-lane pipeline, or a GPU-resident path and a dedicated streaming block. Alternatives must differ in a meaningful architectural tradeoff, not only their labels.

For each, draw dataflow, storage, transfers, queues and ownership. State precision and compute/memory ceilings. Estimate setup, result-consumption and synchronization costs, then compute the maximum application benefit using the original workload fraction and critical path. Explain which resource saturates first as batch size grows.

For a bounded model, count only what it actually represents. “Four arithmetic lanes” is a modeled structure, not a gate count. Assumed 100 MHz timing is not clock closure. A cache-free model cannot establish real cache hit rates; a deterministic stall pattern does not establish a physical memory controller's distribution.

Numerical verification

Define whether equality is bit-exact, absolute-tolerance, relative-tolerance or application-quality based. State treatment of zero, negative values, ties, extremes, NaN/infinity where relevant, overflow and saturation. Explain how intermediate precision and operation order affect results. For authoritative simulation, investigate determinism and threshold decisions rather than relying only on mean error.

Required cases include empty and singleton batches, counts just below/at/above a lane or vector width, extreme representable values, rounding boundaries, saturation boundaries and representative mixed values. A failed case blocks acceptance until fixed or until the contract is legitimately revised and all comparisons repeated.

The reference fixed-point operation uses signed 16-bit raw values with eight fractional bits, product scaling rounded to nearest-even, an encoded bias, then signed saturation. Its independent C++ exact-double oracle is valid for the small exact binary products used by that example; do not assume the same technique is exact for arbitrary expanded formats.

Host/device interface and failure plan

Write the descriptor schema, strides, alignment, bounds, registered memory, operation, format, request identity and context. Define ownership states and the event that permits reuse. Distinguish a host timeout from confirmed hardware quiescence. Epoch changes reject stale logical completions but do not physically stop DMA.

Required failure cases are malformed descriptor, wrong bounds or alignment, duplicate/stale completion, timeout before completion, removal/reset with work pending, and a bounded fallback or rejection. Add wrong-context access when contexts are part of the architecture. A single-buffer model does not establish isolation between tenants or virtual machines.

The supplied descriptor lesson uses four-byte elements; the affine teaching stream uses two-byte inputs and two-byte outputs. Adapt these contracts explicitly before integration. The provided pipeline, descriptor exercise and analytical cost model are separately tested teaching components, not a real driver controlling hardware.

Evaluation matrix

Evaluate every candidate at the same useful boundary. Separate functional evidence, model estimates and host/device measurements. The minimum matrix follows; add workloads that distinguish your hypothesis from competing explanations.

Dimension	Minimum cases	Why it matters
Batch size	Empty, singleton, tail, typical, large	Correctness, startup and amortization
Input distribution	Typical, boundary, adverse	Quality and divergent behavior
Readiness	Normal and backpressured/delayed completion	Ordering, loss and lifetime
Memory	Nominal and tighter bandwidth/capacity	Dominant resource and feasibility
Timing	Setup, compute, transfer, completion, end-to-end	Honest useful-result boundary
Lifecycle	Timeout, reset, removal or equivalent	Bounded safe recovery

For each measured series retain raw data, metadata, warmup policy, repetitions, order and exclusions. Label percentiles and sample counts. A timing unit of nanoseconds is not a claim of nanosecond accuracy. Include a fresh-session repeat or an independently reproduced run. A single lucky minimum is insufficient for an interactive-tail claim.

For modeled series provide equations, constants, units, code configuration and conditional conclusions. Compare model trends with any accessible measurements without claiming the measurements validate omitted hardware behavior.

Sensitivity and application benefit

Sweep at least three uncertain or design-sensitive parameters, including setup/transfer cost and a memory or latency constraint. Use a low/nominal/high table tied to plausible or explicitly hypothetical values. Explain where the preferred design changes, not just whether the nominal point looks good. If the model is insensitive to a parameter, show the calculation rather than omitting the parameter.

Compute the maximum possible application saving even with infinite acceleration of the candidate operation. Then estimate the achievable saving with overhead and unchanged work. For overlapping execution, use a dependency timeline instead of a naïve sum. Answer what happens when memory capacity is tightened, when available bandwidth falls and when the latency budget is reduced.

A recommendation should identify the dominant bottleneck, crossover batch sizes, quality limitations, maintenance/verification cost and next measurement that would most reduce uncertainty. It may recommend optimized software, a narrower accelerator or no change.

Milestones and review artifacts

M1: workload dossier, requirements and initial maximum-benefit calculation. Review whether the operation is worth studying before implementation.

M2: trusted oracle, competent baseline, raw measurement protocol and intentional-wrong-result rejection. Review fairness and timing boundaries.

M3: two architecture alternatives with dataflow and estimates. Implement one bounded specialized path and its numerical and flow-control tests.

M4: adapted host/device contract, malformed-request and lifecycle tests, end-to-end evaluation across the required matrix.

M5: sensitivity study, evidence ledger, independent reproduction and final recommendation. Record review objections and revisions; preserve unsupported claims that were withdrawn in the revision log rather than silently replacing their evidence.

These milestones fit inside Module 40's planning allocation and reuse prior module artifacts. They are not five extra unrelated projects.

Suggested portfolio layout

```
portfolio/
  README.md
  requirements.md
  workload/                # authorized data and provenance
  baseline/                # source, build settings, oracle
  alternatives/            # two architectures; one implemented
  interface/               # descriptors, ownership, failure policy
  tests/                   # numerical, tails, lifecycle and faults
  raw_results/             # unedited observations
  analysis/                # equations, scripts and summaries
  evidence_ledger.json
  review.md
  recommendation.md
```

README must provide exact reproduction commands and dependencies, distinguish required and optional tools, and state which files are generated. A reviewer should regenerate the main tables from raw data without hand-editing numbers. Keep private SDKs, credentials and unauthorized assets out of the portfolio.

Rubric and mandatory acceptance conditions

Category	Points	Full-credit evidence
Workload and requirements	15	Measured relevant distribution, explicit quality/boundaries
Baseline and oracle	15	Fair implementation, validated outputs, reproducible method
Alternatives and implementation	20	Two credible alternatives and one tested specialized path
Interface and failure behavior	15	Bounds, ownership, cancellation/ quiescence and adverse tests
End-to-end evaluation and sensitivity	20	Complete costs, multiple scales, uncertainty and tighter constraints
Evidence and design review	15	Correct tiers, provenance, independent reproduction and clear decision

A suggested readiness threshold is 80/100 after revision. This local rubric is not an accredited examination. Regardless of score, the portfolio cannot pass with a known unresolved correctness/ ownership defect, unsupported physical claims, an intentionally weak baseline, missing measured workload, missing second alternative or irreproducible principal results. Reviewers may require additional cases when the claimed scope exceeds the minimum matrix.

Negative performance does not automatically lose points. A rigorously demonstrated loss can earn full credit when implementation, tests and reasoning meet the requirements. Missing evidence is different from negative evidence.

Oral review prompts

Explain the strongest reason the proposal could be wrong. Identify the exact memory level used by the roofline model. Show a tail and rounding-boundary test. Trace one request through timeout and quiescent reset. Explain the difference between kernel throughput and input-to-display latency. Identify the maximum application benefit and the constraint that changes your preferred architecture. Show the file and method behind one major numerical claim.

A reviewer should be able to choose an adverse case, rerun it and see the stated behavior. A memorized presentation without working evidence does not complete the capstone.

Boundaries of the supplied reference

The included reference runs 36 functional/model combinations across six sizes, three lane counts and normal/adverse readiness. The native option measures two host C++ variants. It records all model assumptions, verifies results and writes an auditable ledger. See REFERENCE_CASE_STUDY and the raw result folder.

It is not a real engine workload, complete learner portfolio, GPU benchmark, FPGA bitstream, synthesis report, physical board test, product power measurement or console certification. Use it to learn the workflow and expose mistaken comparisons. Completing Gate H requires your own workload connection, requirements, adaptations, sensitivity and review.